

GOVT 10: Setup Guide (Windows)

Installing R, Positron, Claude, and GitHub Desktop on Windows

Sean Westwood

Table of contents

1 Before you begin	1
2 Install R	2
3 Install Positron	2
3.1 First-run setup	3
4 Set up Claude	3
4.1 Subscribe to Claude Plus	3
4.2 Install Claude Code	4
4.3 Connect Claude Code to your account	4
5 Create a GitHub account	4
5.1 Sign up	4
5.2 Apply for the GitHub Student Developer Pack	5
5.3 Install the GitHub Desktop app	5
5.4 Sign in and configure	5
5.5 Clone the class repo	5
5.6 Run the course setup script	6
6 Putting it all together	6
7 Quick reference	7

1 Before you begin

Set aside about **45 minutes** in a quiet place with a reliable internet connection. You will install four tools and create one account. None of this is technically hard, but small mistakes early can become big headaches later. Read each step before clicking.

What you will install

1. **R** — the statistical programming language

2. **Positron** — the editor we will use to write and run R code
3. **Claude** — the AI assistant; you will use *Claude Plus* on `claude.ai` and *Claude Code* in PowerShell
4. **GitHub Desktop** — the app that handles Git and pushes your work to GitHub
5. **GitHub account** — where you will store and share your work

Order matters. Install in the order listed above. Several tools look for the others during their own installation; if R isn't on your machine when Positron starts, Positron won't find it later without extra steps.

Do these checks first:

- Your PC must have at least **8 GB RAM** and **15 GB free disk space**.
- Your operating system must be **Windows 10 or Windows 11, 64-bit**.
- You need **administrator rights** on the PC. (If it's a managed work laptop, please switch to a personal device.)
- You will need an **email address** and a way to receive verification codes.

2 Install R

R is the statistical programming language we use to analyze data. Positron uses R under the hood — if R isn't installed, Positron has nothing to run.

1. Open Edge (or any browser) and go to <https://cran.r-project.org/bin/windows/base/>
2. Click the big link at the top: **Download R-4.x.x for Windows**.
3. Open the downloaded `.exe` file in your Downloads folder. Windows may show a “Did this app come from a trusted source?” warning — choose **More info** then **Run anyway**. CRAN is the official R distribution and is safe.
4. Step through the installer. Accept the defaults at every screen — do not change the install path.
5. When the installer finishes, close it. **You will never open R directly**. Positron will start R for you.

3 Install Positron

Positron is the editor we will use all term. It looks similar to VS Code (if you've seen that) but is purpose-built for R and data work. It is free and made by the same team that builds RStudio.

1. Go to <https://positron.posit.co/>
2. Click **Download for Windows**. You'll get a `.exe` installer in your Downloads folder.

3. Open the `.exe`. Windows SmartScreen may say “Windows protected your PC” — click **More info** then **Run anyway**.
4. Step through the installer. When you reach the “Select Additional Tasks” screen, **check the box for “Add Positron to PATH”** and **“Create a desktop shortcut”**.
5. Finish the installer. Open Positron from the Start menu or the desktop shortcut.

3.1 First-run setup

1. When Positron opens for the first time, it shows a Welcome page.
2. Positron will automatically find your R installation and offer to start a fresh R session. Click **Start R**. If you see “No R installation found”, restart your PC and reopen Positron — this usually happens when R was installed in the same session and the PATH hasn’t refreshed.
3. In the bottom panel you should see an **R Console** with a prompt that looks like `>`. Type:

```
2 + 2
```

and press Enter. You should see `[ 1 ]` 4. R is working.

Positron is installed when `2 + 2` returns `[ 1 ]` 4 in the R Console and Positron starts cleanly.

Leave Positron open. We will come back to it after cloning the class repo and running the setup script.

4 Set up Claude

We use Claude two different ways in this class. Both require a **paid Claude Plus subscription** (\$20/month). Dartmouth does not provide a license — this is a personal subscription.

4.1 Subscribe to Claude Plus

1. Go to <https://claude.ai/>
2. Click **Sign up**. Use your **Dartmouth email address** so your work is tied to your academic identity. Verify the email when Claude sends a code.
3. Once signed in, click your initials in the top-right → **Settings** → **Plan**.
4. Choose **Plus** (\$20/month). Enter payment information. The subscription is month-to-month — you can cancel after the term ends.

You now have access to Claude on the web at `claude.ai`. Bookmark it.

4.2 Install Claude Code

Claude Code is a command-line tool that talks to your project files. We use it for coding tasks during class.

1. Press the **Windows key**, type `powershell`, then **right-click** “Windows PowerShell” → **Run as administrator**. Click **Yes** if Windows asks for permission.
2. Paste this line and press Enter:

```
irm https://claude.ai/install.ps1 | iex
```

3. When the installer finishes, **close and reopen PowerShell** so the new command becomes available.

4.3 Connect Claude Code to your account

1. In PowerShell, type `claude` (just the word, by itself) and press Enter.
2. The first run prints a URL. Open it in your browser, sign in with the same account you used at `claude.ai`, and authorize the CLI.
3. PowerShell will say “Authentication successful”. Press **Ctrl+C** to exit.

Keep your account credentials private. Never share your Claude API keys, session tokens, or login. We will use Claude in class — when prompted, sign in fresh on the classroom machine, do not save credentials.

5 Create a GitHub account

GitHub is where you will store your code, push your assignments, and (if you want) build a portfolio for future internships and jobs.

5.1 Sign up

1. Go to <https://github.com/>
2. Click **Sign up** in the top-right.
3. Use the **same Dartmouth email** you used for Claude. Choose a username that is professional — it appears on every repository you create and is hard to change later. Many people use a variant of their real name.
4. GitHub will email a verification code. Enter it.
5. Pick the **free plan** when asked.

5.2 Apply for the GitHub Student Developer Pack

The Student Pack gives you free access to private repositories, GitHub Copilot, and dozens of other developer tools.

1. While signed in, go to <https://education.github.com/pack>
2. Click **Get your pack**.
3. Verify your status as a Dartmouth student. The fastest way is to **upload a photo of your Dartmouth ID** or use your `@dartmouth.edu` email — both usually approve within a day.

5.3 Install the GitHub Desktop app

GitHub Desktop is the official desktop app for GitHub. It installs Git under the hood (so Positron will find Git automatically), lets you clone, commit, and push through buttons rather than the terminal, and handles sign-in once for the whole machine.

1. Go to <https://desktop.github.com/>
2. Click **Download for Windows**. Save the `.exe` to your Downloads folder.
3. Open the `.exe`. The app installs and launches automatically. (If Windows SmartScreen warns you, choose **More info** → **Run anyway**.)

5.4 Sign in and configure

1. On the welcome screen of GitHub Desktop, click **Sign in to GitHub.com**.
2. Your browser opens. Click **Authorize desktop**. Approve the access request.
3. The browser will ask to “open GitHub Desktop?” — click **Open**.
4. Back in the app, you will be asked to **Configure Git**. It pre-fills your name and email from your GitHub account. Confirm both — this is the identity that will appear on every commit you make.
5. Click **Finish**.

You're set up. GitHub Desktop's main window shows “No repositories found” — that's expected. We'll clone the course repo next.

5.5 Clone the class repo

The course materials — lectures, exercises, data, the setup script — all live in a GitHub repository. Clone it once and GitHub Desktop will keep your local copy in sync.

1. In GitHub Desktop, click **File** → **Clone repository...** (or **Clone a repository from the Internet...** on the welcome screen).
2. Switch to the **URL** tab.
3. Paste the class repo URL (your instructor will provide this — check the syllabus or course site).

4. For “Local path”, click **Choose...** and pick a folder where you’d like the materials to live. Your **Documents** folder is a good default — GitHub Desktop will create a subfolder for the repo.
5. Click **Clone**. The download takes a minute or two.
6. When it finishes, GitHub Desktop shows the repo as your “Current repository” at the top-left.

Where did the files go? GitHub Desktop tells you. Click **Repository** → **Show in Explorer** to open the local folder. You should see week folders, the syllabus, and `setup.R` sitting at the top level.

5.6 Run the course setup script

Now that the repo is on your PC, run the setup script that came with it.

1. In Positron, click **File** → **Open Folder...** and choose the folder GitHub Desktop just cloned (e.g. `Documents\govt10-materials`). Opening the folder (not just the file) tells Positron the repo is your “project” for the term.
2. In the **Explorer** panel on the left, click **setup.R** to open it in the editor.
3. Press **Ctrl+A** to select the whole file, then **Ctrl+Enter** to run the selection.
4. The script will print progress messages in the R Console as it downloads packages. **This takes 10–15 minutes.** Don’t close Positron; let it finish.
5. When the Console shows the prompt `>` again with no red error text, you’re done.

If the script stops with a red error, copy the last 10 lines of red text and email me. The most common failure is a network timeout — in that case, simply rerun the script and it will resume where it stopped.

You can still use Positron for Git. Positron’s built-in Source Control panel (the branching-diagram icon in the left activity bar) uses the same Git that GitHub Desktop installed. Many people prefer to commit/push from inside the editor; both approaches work. Pick whichever feels easier.

6 Putting it all together

A typical workflow during class will look like this:

1. Open **Positron**. Open the project folder for the week.
2. Write R code in the editor pane, run it in the R Console.
3. When stuck on syntax or a concept, open **Claude** (web or Claude Code) and ask. Always read what Claude returns and verify it against the lecture.
4. Save your work, then commit and push with **GitHub Desktop** (or Positron’s Source Control panel).

You don't need to be an expert in any of these tools on day one. Class time is where you build that muscle memory.

Help, something broke!

- **Restart your PC.** Half of "installation didn't work" problems resolve after a reboot — PATH variables and helper services need a clean start.
- **Read the exact error message.** Don't paraphrase; copy it.
- **Email me before class with the error.** Subject line: "GOVT 10 setup: [tool name]". Attach a screenshot. I will reply quickly.
- Office hours (Wednesday 4:30–6:00 PM EST, by appointment) are the right time for any setup issues that survive these steps.

7 Quick reference

Tool	What it does	Where you got it
R	Statistical language	cran.r-project.org/bin/windows/base/
Positron	Code editor for R	positron.posit.co
Claude Plus	AI assistant (web)	claude.ai
Claude Code	AI assistant (PowerShell)	claude.ai install script
GitHub	Stores code online	github.com
GitHub Desktop	Clones / commits / pushes	desktop.github.com
GitHub Student Pack	Free pro features	education.github.com/pack

Before the first class: make sure all the tools above are installed. You should be able to:

- Open Positron and see `[ 1 ] 4` after running `2 + 2` in the R Console.
- Run `library(tidyverse)` in Positron without red errors (this confirms `setup.R` succeeded).
- Sign into `claude.ai` in the browser.
- Open GitHub Desktop and see your GitHub username in the top-right.

If any of those checks fails, email me **before** class so we can fix it in advance.